Random Forests

2026-04-17

Introduction

Random forests, introduced by Breiman in 2001, average predictions from many bootstrapped decision trees, each trained with a random subset of features considered at every split. Two sources of randomness — bootstrap row sampling and feature subsampling — decorrelate the trees and dramatically reduce variance compared with a single tree. The result is a strong off-the-shelf baseline for tabular data: minimal hyperparameter tuning, robust to outliers and irrelevant features, and providing free out-of-bag (OOB) error estimation as a side effect of the bootstrap.

Prerequisites

A working understanding of decision trees, bagging (bootstrap aggregating), and the bias-variance trade-off in ensemble methods.

Theory

For each of B trees:

1. Draw a bootstrap sample (sampling with replacement, same size as original) from the training data.
2. Grow a deep tree on this sample; at each split, consider a random subset of m features (typically $m = \sqrt{p}$ for classification, $m = p/3$ for regression).
3. Predict on each tree; aggregate predictions by averaging (regression) or majority voting (classification).

Out-of-bag (OOB) error uses observations not selected by each bootstrap as a held-out validation set; aggregating these per-tree predictions across the forest gives a nearly unbiased generalisation estimate at zero additional computational cost.

Assumptions

Observations are independent and identically distributed; features are informative (random forests are robust to irrelevant features but cannot synthesise signal that is not there); the default m value is a starting point but may need tuning for specific datasets.

R Implementation

```
library(ranger)

set.seed(2026)
n <- 500
df <- data.frame(x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n),
                 y  = factor(sample(c("pos", "neg"), n, replace = TRUE)))
df$y <- factor(ifelse(plogis(0.8 * df$x1 - 0.6 * df$x2) > runif(n), "pos", "neg"))

rf <- ranger(y ~ ., data = df,
             num.trees = 500,
             mtry = 1,
```

```
             importance = "impurity",
             classification = TRUE)

rf$prediction.error         # OOB misclassification error
rf$variable.importance
```

Output & Results

`ranger()` returns a fitted random forest with OOB error (`prediction.error`), per-feature variable importance (`variable.importance`), and the underlying tree structures. The OOB error is the standard generalisation estimate; variable importance ranks features by their average contribution to prediction quality.

Interpretation

A reporting sentence: "A 500-tree random forest with `mtry = 1` achieved OOB classification error of 0.17; variable importance identified x_1 (impurity decrease 32) and x_2 (decrease 18) as the top predictors, consistent with the data-generating logistic process. A held-out test set confirmed the OOB-based estimate (test error 0.16)." Always report OOB error or held-out test error and the top variable importances.

Practical Tips

- Use `ranger` instead of the original `randomForest` package for new analyses; it is dramatically faster and lower-memory while producing essentially identical results.
- Tune `mtry` on 5-fold cross-validation; the default ($\sqrt{p}$ for classification, $p/3$ for regression) is reasonable but not always optimal.
- `min.node.size` controls leaf size; larger values give more bias and less variance — useful for very noisy datasets.
- For unbiased variable importance, use `importance = "permutation"` instead of `"impurity"`; it is slower but less biased toward continuous features and high-cardinality categoricals.
- OOB error is nearly unbiased for moderate to large n; use it instead of cross-validation for rapid hyperparameter screening, then validate the final choice on a held-out test set.
- For class imbalance, use `class.weights` or `case.weights` (in `ranger`); both are natively supported and more efficient than over- or under-sampling.

R Packages Used

`ranger` for fast random forests with OOB and permutation importance; `randomForest` for the original Breiman implementation; `parsnip::rand_forest()` for the tidymodels-flavoured wrapper; `caret` for cross-validated tuning; `vip` for variable-importance visualisation; `pdp` for partial-dependence plots.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting

- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis